

人工智能课程设计实验报告

八皇后问题



学 号 2351520

姓 名 毛星博

专 业 基础学科拔尖基地(计算机)

授课老师 武妍

一、问题概述

1.1 八皇后问题的描述

八皇后问题是一个经典的组合优化问题，最早由国际象棋棋手马克斯·贝瑟尔（Max Bezzel）于 1848 年提出。这个问题要求在一个 8×8 的国际象棋棋盘上放置 8 个皇后，使得没有两个皇后能够互相攻击。根据国际象棋的规则，皇后可以攻击同一行、同一列或同一斜线上的任何棋子。也就是说，满足这个问题的解必须满足以下条件：

1. 任意两个皇后不能位于同一行
2. 任意两个皇后不能位于同一列
3. 任意两个皇后不能位于同一对角线上（包括主对角线和副对角线）

1.2 八皇后问题的研究历史

该问题在 1848 年由国际象棋棋手马克斯·贝瑟尔首次提出。

第一个解在 1850 年由弗朗兹·诺克（Franz Nauck）给出。并且将其推广为更一般的 n 皇后摆放问题。诺克也是首先将问题推广到更一般的 n 皇后摆放问题的人之一。

在此之后，陆续有数学家对其进行研究，其中包括高斯和康托。高斯计算出 76 种解法（后修正为 92 种）。1854 年多个研究者发表不同解法，最终通过图论方法确认存在 92 种有效布局。

计算机发明后，该问题成为回溯算法、递归算法的经典案例，并衍生出对解法的分类研究。考虑棋盘旋转（ 90° 、 180° 、 270° ）和镜像对称后，等价解归并为 12 种基本模式。

1.3 八皇后问题的求解思路

本次实验采用 Python 的 kanren 库进行逻辑编程，通过约束逻辑的方式解决八皇后问题。解决问题的关键在于满足问题的几个约束条件：任意两个皇后不能在同一行、同一列、同一对角线。

一开始，我尝试使用一个 8×8 的数组来储存每个皇后的位置，但是这样需要考虑的约束条件太多，导致代码非常复杂。

后来我发现，既然每行每列只能有一个皇后，那么我只需要用一个长度为 8 的数组来表示一个解，数组中元素的位置表示皇后所在的行，而元素的值表示皇后所在的列。

这样，我只需要考虑一个约束条件：任意两个皇后不能在同一对角线。

而逻辑编程的思维使我可以不用考虑如何实现这个约束条件，只需要将这个约束条件表达出来，然后遍历所有可能的解，找到满足约束条件的解即可。

那么所有的解是什么呢？

思考问题的本质，既然每行每列只能有一个皇后，也就是说，每个数字只能出现一次。由此可以发现，每个可能的解其实就是这 8 个数字的一种排列，其中第 i 个元素表示第 i 行皇后所在的列。

那么，我只需要生成 1-8 的全排列，然后检查每个排列是否满足约束条件，如果满足，则这个排列就是一个解。由于每一种排列表示的解已经自动满足行列的约束，所以需要检查

的约束条件只有一个——对角线的约束。这就大大简化了约束逻辑！

因此，最终的解决思路为：

首先将八皇后问题转化为寻找 1-8 数字的全排列问题。每个排列对应一种候选解，其中索引表示行号，数值表示列号。利用逻辑编程的约束求解特性，只需生成 1-8 的全排列，添加对角线约束，最后通过 `kanren` 的求解器自动推导满足上述约束的组合。

二、算法设计

2.1 算法功能

本算法旨在解决八皇后问题，即在 8×8 的国际象棋棋盘上放置 8 个皇后，使得没有两个皇后能够互相攻击。算法通过逻辑编程的方式，利用约束条件筛选出所有可能的解，并以可视化的方式展示这些解。

2.2 设计思路

我采用了基于约束逻辑的方法，结合排列生成和约束满足技术来解决八皇后问题。

具体思路如下：

1. 问题表示：

使用一个长度为 8 的元组表示一个解，其中元组的索引表示行号，元素值表示列号

例如，元组(0,4,7,5,2,6,1,3)表示在第 0 行第 0 列、第 1 行第 4 列、第 2 行第 7 列等位置放置皇后。因为每一个元组都是八个数字的一种排列，所以这种表示方法自动满足了"每行只有一个皇后"的约束。

2. 约束条件：

对角线约束：通过检查函数确保没有皇后在同一条对角线上。而对角线约束的判断方法，在数学上考虑也很简单，只需两个皇后的行差和列差的绝对值不相等。即：对于任意两个位置(i,vi)和(j,vj)，需满足 $|i-j| \neq |v_i-v_j|$ 。

3. 求解：

首先生成所有可能的排列（ $8! = 40,320$ 种）

然后使用 `kanren` 的约束求解机制筛选出满足对角线约束的解

4. 结果展示：

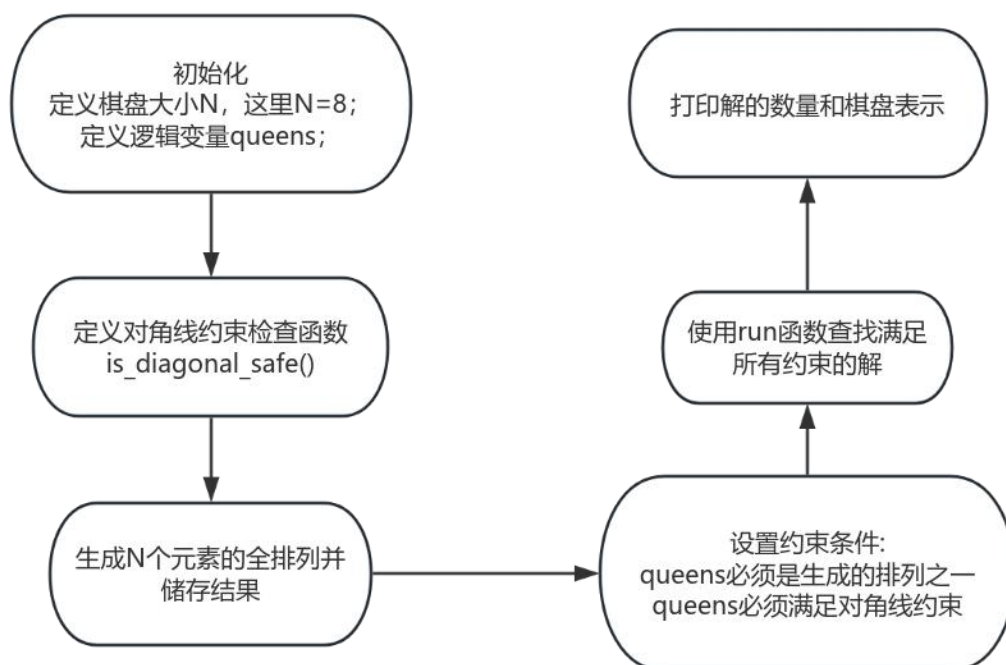
首先打印找到的解的数量

然后可视化每个解的棋盘布局

2.3 算法流程

1. 定义对角线约束检查函数 `is_diagonal_safe()`
2. 生成所有可能的排列：生成 0 到 7 的所有排列，并将生成的排列存储
3. 设置约束条件：满足对角线约束：通过检查函数确保没有皇后在同一对角线上
4. 求解：使用 `kanren` 的 `run` 函数查找满足所有约束的解
5. 结果展示：打印找到的解的数量，并可视化每个解的棋盘布局

2.4 算法流程图



三、算法的实现

3.1 数据结构的定义

本题只需定义问题的规模大小 `N`（这里 `N=8`），和逻辑变量 `queens`（8 个元素的元组，实际为 8 个数字的一种排列，表示问题的一个可能的解）

```
# 定义棋盘大小，八皇后问题为8
N = 8
# 定义逻辑变量
queens = var("queens")
```

3.2 导入 kanren 库中的必要函数

```
from kanren import run, eq, membero, var, goalify
```

`var()`: 生成一个逻辑变量（即未知量），用于表示待求解的抽象值。

`run()`: 执行逻辑求解。根据给定的约束条件搜索解，返回满足条件的变量绑定结果。

参数: `run(n, vars, goals)`, 其中:

`n`: 返回解的数量（0 表示所有解）。

`vars`: 需要求解的变量或变量列表。

`goals`: 约束条件（目标集合）。

`eq(a, b)`: 等式约束。声明两个变量或值必须相等，生成一个逻辑目标。

`membero(x, coll)`: 成员约束。声明变量 `x` 必须是集合 `coll` 中的某个元素。

`goalify(func)`: 将普通 Python 函数转换为可在逻辑编程中使用的约束目标。

3.3 核心函数

3.3.1 对角线约束检查函数 `is_diagonal_safe`

```
def is_diagonal_safe(queens: tuple):
    """检查皇后放置是否满足对角线约束（没有皇后在同一对角线上）"""
    for i in range(N):
        for j in range(i + 1, N):
            if i + queens[i] == j + queens[j] or i - queens[i] == j - queens[j]:
                # 皇后在同一对角线上，返回 False
                return False
    return True
```

这个函数通过检查任意两个皇后是否在同一对角线上来实现对角线约束。具体来说:

`i + queens[i] == j + queens[j]`: 检查两个皇后是否在同一正对角线上

`i - queens[i] == j - queens[j]`: 检查两个皇后是否在同一负对角线上

3.3.2 全排列生成函数 `generate_permutations`

```
def generate_permutations(elements):
    """生成元素的所有排列"""
    if len(elements) == 0:
        yield ()
    else:
        for i in range(len(elements)):
            # 取出当前元素
            current = elements[i]
            # 剩余元素
            remaining = elements[:i] + elements[i+1:]
            # 对剩余元素生成排列，并在每个排列前加上当前元素
            for p in generate_permutations(remaining):
                yield (current,) + p

# 生成所有可能的排列，i 表示第 i 行，queens[i] 表示第 i 行皇后所在的列
```

```
all_permutations = tuple(generate_permutations(tuple(range(N))))
```

这个函数使用递归方式生成给定元素的所有排列，当元素列表为空时，生成一个空元组 `()`，表示唯一可能的排列，这是递归的基线条件。当不满足基本条件时，通过递归将问题分解为更小的子问题。

- 1: 遍历每个元素 `current`，将其从列表中“固定”为当前排列的第一个元素。
- 2: 对剩下的元素 `remaining` 递归调用 `generate_permutations`，生成所有可能的子排列。
- 3: 将 `current` 与每个子排列 `p` 组合，形成完整排列。

3.3.3 逻辑求解函数 `run`

```
# 找到满足所有约束的解
solutions = run(
    0, # 0 表示找到所有解
    queens,
    membero(queens, all_permutations), # queens 必须是 all_permutations 中的一个元素
    (goalify(is_diagonal_safe), (queens,), True), # 对角线检查必须返回 True
)
```

1. 参数 `0` 表示返回所有可能的解
2. `membero(queens, all_permutations)` 强制 `queens` 的列位置组合必须是 `all_permutations` 中的一个全排列。`all_permutations` 是通过 `generate_permutations` 生成的 $8!$ 种排列（如 `(0,1,2,3,4,5,6,7)` 的排列组合），确保皇后列位置满足每行每列唯一性。
3. `(goalify(is_diagonal_safe), (queens,), True)` 将自定义函数 `is_diagonal_safe` 转换为逻辑编程目标，要求目标函数返回 `True` 才能满足约束。

3.4 模块的输入和输出

本算法不需要外部输入，棋盘大小 `N=8` 是固定的；所有可能的排列是在程序内部生成的。

输出：

1. 解的数量：打印找到的解的总数（应为 92 个）
2. 解的可视化表示：以棋盘形式打印每个解，其中“Q”表示皇后，“.”表示空位

```
print(f"八皇后问题的解：共找到 {len(solutions)} 个解")
print("\n 棋盘表示 (Q 表示皇后，. 表示空位):")
```

```
# 打印棋盘，Q 表示皇后，. 表示空位
def print_board(solution):
    """打印棋盘"""
    for i in range(N):
        row = ""
        for j in range(N):
            if solution[i] == j:
                row += "Q "
            else:
                row += ". "
        print(row)
```

```

        print(row)

# 打印所有解的棋盘
print(f"\n 以下是所有 {len(solutions)} 个解:")
for i, solution in enumerate(solutions):
    print(f"\n 解 #{i+1}:")
    print_board(solution)

```

四、实验结果

本次实验成功解决了八皇后问题。程序计算出了 92 种正确的解，并将每种解可视化。

`len(solutions)`表示解的总数，一共 92 种解，是正确答案捏~

可视化打印棋盘，Q 表示皇后，.表示空位

```

C:\WINDOWS\System32\cmd.  x  +  v
八皇后问题的解：共找到 92 个解
棋盘表示（Q表示皇后，.表示空位）：
以下是所有 92 个解：

解 #1:
Q . . . . . .
. . . Q . . .
. . . . . Q
. . . . Q . .
. Q . . . . .
. . . . . Q
. Q . . . . .
. . . Q . . .

解 #2:
Q . . . . . .
. . . . Q . .
. . Q . . . .
. . . . . Q
. Q . . . . .
. . . . . Q
. . . Q . . .
. . . . . Q

解 #3:
Q . . . . . .
. . . . . Q
. . . . . Q
. . . Q . . .
. Q . . . . .
. . . . . Q
. . . . . Q
. . . . . Q

解 #90:
. . . . . . Q
. Q . . . . .
. . . Q . . .
. . . Q . . .
Q . . . . . .
. . . . . Q
. . . Q . . .
. . . . . Q

解 #91:
. . . . . . Q
. . Q . . . .
Q . . . . . .
. . . . . Q
. Q . . . . .
. . . . Q . .
. . . Q . . .
. . . . . Q

解 #92:
. . . . . . Q
. . . Q . . .
Q . . . . . .
. . Q . . . .
. . . . . Q
. Q . . . . .
. . . . . Q
. . . . . Q

请按任意键继续 . . . |

```

五、总结与分析

5.1 本次实验遇到的问题 and 解决方法

问题 1: 约束条件复杂度爆炸

最初尝试用二维数组表示棋盘时，需要同时处理行、列、主/副对角线 4 类约束，代码中嵌套了多层循环和条件判断，逻辑混乱且难以调试。

解决方法：观察到每行必须且只能放置一个皇后，这意味解的结构本质上是一个排列问

题——8 个列号在 8 个行位置上的排列组合。因此，将二维棋盘映射为一维数组 `queens[i]`，其中索引表示行号，值表示列号，自动满足行列不重复的约束。这样一来，我仅需专注处理对角线冲突，最终将问题转化为 $|i-j| \neq |queens[i]-queens[j]|$ 的数学条件。

问题 2：如何生成全排列

在八皇后问题中，我发现必须遍历所有可能的皇后列位置排列，但如何系统性地生成所有排列成为我的障碍。

解决方法：递归分治生成排列，查阅全排列生成代码的资料，我意识到， n 个元素的全排列可以分解为，将 n 个元素依次放到开头，再和剩下 $n-1$ 个元素的全排列组合。由此，可以得到全排列的递归生成方法：当元素列表为空时，唯一排列是空序列。对于非空列表，逐个固定当前元素，递归生成剩余元素的排列，再将当前元素与子排列合并。

问题 3：逻辑编程的学习

`kanren` 库的使用对我来说是全新的，逻辑编程的思维方式与传统的命令式编程有很大不同。通过阅读文档和示例，我逐渐理解了逻辑编程的核心概念，如逻辑变量、约束和目标。我发现逻辑编程特别适合解决约束满足问题，因为它允许我们以声明式的方式描述问题，而不是命令式地指定解决问题的步骤。

5.2 对算法的理解与评价

优点：

1. 声明式编程：使用逻辑编程范式，代码简洁、易于理解和维护。只需描述问题的约束条件，而不需要指定具体的求解步骤。
2. 问题表示的优化：使用一维数组表示解，自动满足了行和列的约束，大大简化了问题的表述。
3. 完整性和正确性：算法能够找到八皇后问题的所有 92 个解，保证了结果的正确。
4. 可视化：通过 `print_board` 函数，可以直观地展示每个解的棋盘布局，便于理解和验证。

缺点：

1. 可扩展性有限：对于 N 较大的情况，生成所有排列的方法效率较低，时间复杂度为 $O(N! \times N^2)$ ，空间复杂度为 $O(N \times N!)$ 。
2. 内存消耗：存储所有排列需要大量内存，对于 N 较大的情况可能导致内存溢出。

5.3 算法的优化可能

本次实验中，我采用了递归的思路求解全排列问题，在代码复杂度和算法效率上有不足之处。查阅网上资料我得知，其实 `python` 提供了现成的库函数来获取全排列：`from itertools import permutations`。如果通过这种方式，应该可以大大简化代码和提高效率。

5.4 实验的收获与思考

通过本次实验，我意识到问题表示的重要性——选择合适的问题表示方法可以大大简化

问题的求解。在本实验中，使用一维数组表示解而不是二维数组，使得约束条件的表达变得简单明了。

同时，我学习到逻辑编程的价值——逻辑编程提供了一种全新的思考问题的方式，特别适合解决约束满足问题。它允许我们关注"什么是解"而不是"如何找到解"。除了常见的命令式编程和面向对象编程，逻辑编程、函数式编程等范式也有其独特的价值和适用场景。掌握多种编程范式可以帮助我们更灵活地解决各种问题。

总的来说，通过本次实验，我不仅学习了解决八皇后问题的方法，还深入理解了逻辑编程的思想和约束满足问题的特点。这些知识和经验将对我未来解决类似问题提供宝贵的参考。

六、附件-源代码

```
from kanren import run, eq, membero, var, goalify

# 定义棋盘大小，八皇后问题为8
N = 8

def is_diagonal_safe(queens: tuple):
    """检查皇后放置是否满足对角线约束（没有皇后在同一对角线上）"""
    for i in range(N):
        for j in range(i + 1, N):
            if i + queens[i] == j + queens[j] or i - queens[i] == j - queens[j]:
                # 皇后在同一对角线上，返回False
                return False
    return True

def generate_permutations(elements):
    """生成元素的所有排列"""
    if len(elements) == 0:
        yield ()
    else:
        for i in range(len(elements)):
            # 取出当前元素
            current = elements[i]
            # 剩余元素
            remaining = elements[:i] + elements[i+1:]
            # 对剩余元素生成排列，并在每个排列前加上当前元素
            for p in generate_permutations(remaining):
                yield (current,) + p

# 生成所有可能的排列，i 表示第 i 行，queens[i] 表示第 i 行皇后所在的列
all_permutations = tuple(generate_permutations(tuple(range(N))))
```

```

# 定义逻辑变量
queens = var("queens")

# 找到满足所有约束的解
solutions = run(
    0, # 0 表示找到所有解
    queens,
    membero(queens, all_permutations), # queens 必须是 all_permutations 中的一个元素
    (goalify(is_diagonal_safe), (queens,), True), # 对角线检查必须返回 True
)

print(f"八皇后问题的解：共找到 {len(solutions)} 个解")
print("\n 棋盘表示 (Q 表示皇后，.表示空位):")

# 打印棋盘，Q 表示皇后，.表示空位
def print_board(solution):
    """打印棋盘"""
    for i in range(N):
        row = ""
        for j in range(N):
            if solution[i] == j:
                row += "Q "
            else:
                row += ". "
        print(row)

# 打印所有解的棋盘
print(f"\n 以下是所有 {len(solutions)} 个解:")
for i, solution in enumerate(solutions):
    print(f"\n 解 #{i+1}:")
    print_board(solution)

```